Browser lifetime + stop rule: branch diff summary (release candidate `9a1b931`)

- Branch: `fix/browser-lifetime-and-stop-rule` (pushed to `origin`)
- Base: `b676c2d`, the head of `fix/worker-isolation`, which records the Railway Trial 1 result
- **Release candidate (code + tests): `9a1b931`.** It adds the reviewer's release items (durable stop-rule cancel, integration-process identification) on top of `caaa191`, whose product code is identical to the accepted fix `f2288ce`. The commits that add these docs, the patch and the evidence sit on top of `9a1b931` and change no code.
- `fix-browser-lifetime-and-stop-rule.patch` = `git format-patch b676c2d..9a1b931 --stdout -- . ':(exclude)deliverables'`, 4 code commits. Applied to `b676c2d`, it reproduces the `9a1b931` tree outside `deliverables/` (checked in a throwaway worktree). A patch cannot contain the commit that adds it.
- No merge, no deploy, no settings/.env change, no paid run. Scoring rules and BM25 are unchanged. No schema change.

Feature gates — what deploying activates

Change	Active when	Effect of deploying to production as configured today
Close+reap the child's Chromium before returning the browser permit	Isolated job children (`isIsolatedChild()`), i.e. `WORKER_ISOLATION_ENABLED=true`	**Active immediately.** Production already runs with `WORKER_ISOLATION_ENABLED=true`, so there is no off switch for the fix short of reverting the deploy. (The in-process `releaseBrowserSlot()` path is unchanged, apart from returning a resolved Promise that callers `await`.)
Batch stop rule S4	`BATCH_STOP_RULE_FAILED_JOBS` = N > 0 (isolated and in-process worker paths)	**Inert** until the variable is set. If it is unset, 0 or invalid, the retry path is the legacy one; test 9 checks this.

Rollback.

- Redeploy the previous production image/commit (`b676c2d`, or the prior production deploy as recorded in the deploy history).
- To turn off only the stop rule: unset `BATCH_STOP_RULE_FAILED_JOBS` or set it to 0.
- Do **not** disable `WORKER_ISOLATION_ENABLED` as the rollback; reverting the deploy is the rollback.

Commits since base (code)

- `9a1b931` **fix(stop-rule): durable-first cancel with bounded purge/snapshot; mark stop-rule snapshot cancelled/partial** (+ real-PG stop-rule test, integration descendant identities)
- `caaa191` test(browser-lifetime): per-document marker extraction, no-skip check, admission wait / permit hold / wall time in repro summary
- `f2288ce` **fix(worker-isolation): close+reap the child's Chromium before returning the browser permit; opt-in batch stop rule S4**
- `f3825cf` evidence(browser-lifetime): before/after repro runs (unlimited + TasksMax=400) with corrected per-profile browser counter (superseded by `evidence/browser-lifetime-candidate/`)
- `979db79` test(browser-lifetime): repro harness + fixture; count one browser per distinct puppeteer profile

Code/test files changed (`git diff --stat b676c2d 9a1b931`, excluding deliverables/)

```
server/lib/__fixtures__/browser-close-order-child.ts |  40 ++++
server/lib/__fixtures__/browser-lifetime-child.ts    |  43 ++++
server/lib/__fixtures__/marker-pdf.ts                |  27 +++
server/lib/batch-stop-rule.db.test.ts                | 159 ++++++++++++++
server/lib/batch-stop-rule.test.ts                   | 238 ++++++++++++++++++++++
server/lib/batch-stop-rule.ts                        | 184 ++++++++++++++++
server/lib/browser-close-order.test.ts               | 150 ++++++++++++++
server/lib/job-isolation.integration.test.ts         |  49 ++++-
server/lib/lifecycle-fence.db.test.ts                |   2 +-
server/lib/processor.ts                              | 144 ++++++++++++--
server/scripts/browser-lifetime-repro.ts             | 232 ++++++++++++++++++++++
server/storage.ts                                    |   6 +-
server/worker.ts                                     |  46 ++++
13 files changed, 1299 insertions(+), 21 deletions(-)
```

The product files changed are `processor.ts`, `worker.ts`, `storage.ts` (`cancelBatchRun` now returns whether it transitioned the batch; 3 lines) and the new `batch-stop-rule.ts`. Everything else is tests, fixtures or the repro script. `lifecycle-fence.db.test.ts` changes one type annotation to match `cancelBatchRun`'s new return type.

What changed

server/lib/processor.ts: browser lifetime in isolated children

Defect. In an isolated child, the browser permit, which the parent brokers per replica up to `MAX_CONCURRENT_BROWSER`, went back to the pool after every fetch. The child's Chromium stayed alive until the child exited, minutes later. So the number of live browsers per replica was bounded by the child count, not by the permit count. Locally that gave 4 live browsers with 1 permit.

Fix

- `releaseBrowserSlot()` now returns a Promise. In an isolated child it calls `releaseChildBrowserPermit()`. When the child releases its **last** held permit, that function does three things in order:
  1. It detaches every Chromium the child launched (`childBrowsers`, `sharedBrowser = null`).
  2. It closes each one (`close()` with a 5 s timeout) and confirms the process group is gone by

scanning `/proc` for non-zombie members of the pgid. If the group survives a 5 s window, it sends SIGKILL to the group and checks again.
3. Only then does it call `remoteRelease("browser")`.

- If there is no pid to probe, only a close that completed counts as confirmed.
- **Unconfirmed cleanup.** The permit is **withheld**, not returned, and `childBrowserRetireFailed` opens the browser circuit for that child, so it cannot deadlock itself. The parent reaps the process when the child exits.
- All three callers await the release, on the normal, refused-after-acquire and finally paths: `fetchWithBrowser`, `fetchPdfViaBrowser` and `fetchIssuerPdfsWithPrimedSession`. `fetchIssuerPdfsWithPrimedSession` also returns early if the circuit is open.
- **Reuse check.** `getSharedBrowser()` in an isolated child uses `browser.connected`. puppeteer-core 25.x has no `isConnected()`, so the old check always failed. The non-isolated check is unchanged.
- `[BROWSER-LIFECYCLE]` log lines record acquire, launch, close, reap and release.

server/lib/batch-stop-rule.ts and server/worker.ts: stop rule S4

Rule. Validation plan §7 rule S4: cancel the batch once ≥ N jobs have failed, or as soon as any job exhausts its retries.

Counting. Each job is counted at its **first** recorded failed attempt. `failed_jobs` only moves at retry exhaustion, which is why batch 1276 let 13 jobs burn 3 attempts each.

Where it runs. In `worker.ts`, on both the failed-result branch and the exception branch, before the retry re-enqueue. The caller skips the retry whenever the rule trips.

Cancel sequence (`runStopRuleCancelSequence`, durable first).

1. `flag`: the local cancelled set + the Redis cross-replica notification (`worker.cancelBatch`), synchronous, so no new claim starts.
2. `pgCancel` (`storage.cancelBatchRun`, the durable `batch_runs.status='cancelled'` write that `claimJob` fences on) and the BullMQ `purge` start **together**. The PG cancel is never gated behind the purge. The purge is bounded by `STOP_RULE_PURGE_TIMEOUT_MS` (10 s); a hang or error is logged and abandoned.
3. Only if the PG cancel committed **and** this call transitioned the batch (`cancelBatchRun` → `true`): the snapshot, bounded by `STOP_RULE_SNAPSHOT_TIMEOUT_MS` (30 s). `saveStopRuleSnapshot` saves the completed companies like `POST /batch/cancel`, then marks every snapshot row of the batch `acceptance_state='rejected'`, `accepted_at=NULL`, `rejection_reason='cancelled/partial snapshot — batch stopped by …; not comprehensive'`.
4. A racing path whose `cancelBatchRun` was the idempotent no-op (`false`) takes no snapshot, so there is no duplicate.

Edge cases

- Cancellations and provider-quota pauses are not counted.
- If counting errors, the rule fails open to the legacy retry path.
- Purge and snapshot errors or hangs are non-fatal and bounded. A PG-cancel error propagates after the bounded purge, with no snapshot taken, and the caller still suppresses the retry.

Review focus

1. `processor.ts` `releaseChildBrowserPermit()` / `retireChildBrowser()`: the ordering, the withheld-permit path and the no-pid case.

2. `worker.ts` `stopRuleTripped()`: where it sits relative to `batchTerminal` and the retry branch.
3. `batch-stop-rule.ts` `runStopRuleCancelSequence()` / `bounded()` and `worker.ts` `saveStopRuleSnapshot()`: durable-first ordering, the timeouts, and snapshot-only-on-transition.
4. `COUNT_BATCH_FAILURES_SQL`: now run against a disposable local PG (unit SQL test, and the real-PG race test `batch-stop-rule.db.test.ts`, 17/17). See `TEST_RESULTS.md` §6.